

Assignment 4

NLP 202: Natural Language Processing II

University of California Santa Cruz

Winter 2023

This assignment is to be done in Python 3.

Your final writeup for this assignment, including the problem and the report of the experimental findings of the programming part, should be no more than four pages long. Most of your grade will depend on the quality of the report. You should submit it as a PDF. We *strongly* recommend typesetting your scientific writing using L^AT_EX. Some free tools that might help: Overleaf (online), TexLive (cross-platform), MacTex (Mac), and TexStudio (Windows).

1 Task

The purpose of this homework is to do sequence labeling using a neural model. The task is biomedical named entity recognition (NER) on the BioNLP/NLPBA 2004 corpus [?]. The dataset has 5 types of entities: DNA, RNA, protein, cell_line and cell_type, in the same BIO format we had in A1.

You will need to implement a Bi-LSTM CRF (and its variants) in PyTorch (section ??) and train it to predict the best label sequence using Viterbi decoding. Report your results in each case.

2 Data format and Evaluation

The BioNLP/NLPBA 2004 dataset contains abstracts from the GENIA bio-named entity corpus version 3.02. The dataset has 5 types of entities: DNA, RNA, protein, cell_line and cell_type, in the BIO format, here ‘B’ denotes the beginning, ‘O’ denotes outside, ‘I’ denotes inside of a Named Entity.

There are 3 files: train, dev, and test. The files are in a 2 column format, where each line contains a single token and its tag, separated by a tab character. Sentences are separated by blank lines.

The train set has 18,546 sentences while the dev set has 105 sentences, and the test set has 3,856 sentences.

Here is an example sentence from the dataset:

```
IL-2 B-protein
responsiveness O
requires O
three O
distinct O
elements B-DNA
within O
the O
```

```
enhancer B-DNA
. O
```

You can evaluate your tagger on the dev and test data using the perl evaluation script provided. The dev and test files are in the same format as the training file, but you would ignore the gold labels in the test file. Your output files with the predictions should have the same format, except that instead of gold labels, it would have predicted labels.

To evaluate your model on the dev set, where your output is in dev.output and the gold labeled data is in dev.answers, run:

```
./evalIOB2.pl dev.answers dev.output
```

3 Implementation

1. **BiLSTM-CRF:** Implement a BiLSTM CRF with minibatch training and train it on the training data [?]. See Appendix ?? for help with implementing minibatching. The loss function to be optimized is log loss:

$$-\text{score}_{\theta}(x_i, y_i) + \log \sum_{y' \in Y} e^{\text{score}_{\theta}(x_i, y')} \quad (1)$$

Use the forward algorithm to compute the partition function and the derivative of the log likelihood with automatic differentiation.

You may refer to and use code from the tutorial here as starter code: https://pytorch.org/tutorials/beginner/nlp/advanced_tutorial.html

2. Write a Viterbi decoder and run your model on the dev and test data. You can reuse the Viterbi decoder from the previous assignments(s) that you have implemented or code from the tutorial.
3. **Character CNN with max pooling layer:** In your BiLSTM-CRF, add a character-level CNN embedding layer with maxpooling for each token. Include these character-level embeddings in addition to traditional the word embeddings. [?] (Refer to the slides from the second lecture on CRFs from NLP 201 about character-level CNNs.) These references are just suggestions: you are free to include these embeddings in any manner you see fit.
4. Modify your BiLSTM-CRF model to use a different loss function taking a cost function into account. You can optimize the loss function based on **one** of the following criteria (your choice – note the minus signs in front of first term in these equations. You may implement more than one for extra credit):

- Max margin or SVM loss

$$-\text{score}_{\theta}(x_i, y_i) + \max_{y' \in Y} (\text{score}_{\theta}(x_i, y') + \text{cost}(y_i, y')) \quad (2)$$

- Softmax margin [?]

$$-\text{score}_{\theta}(x_i, y_i) + \log \sum_{y' \in Y} e^{(\text{score}_{\theta}(x_i, y') + \text{cost}(y_i, y'))} \quad (3)$$

- Ramp loss [?]

$$-\max_{y \in Y} \text{score}_{\theta}(x_i, y) + \max_{y' \in Y} (\text{score}_{\theta}(x_i, y') + \text{cost}(y_i, y')) \quad (4)$$

- Soft ramp loss [?]

$$-\log \sum_{y' \in Y} e^{\text{score}_{\theta}(x_i, y)} + \log \sum_{y' \in Y} e^{(\text{score}_{\theta}(x_i, y') + \text{cost}(y_i, y'))} \quad (5)$$

Use the hamming loss as the cost function.

$$cost_{hamming}(\hat{Y}, Y) = \sum_{j=1}^{|Y|} \delta(\hat{y}_j \neq y_j) \quad (6)$$

Multiply the cost function by a constant factor as a hyperparameter which you are free to choose.

Deliverables In the writeup, be sure to fully describe your models and experimental procedure. Provide graphs, tables, charts or other summary evidence to support any claims you make.

1. Report the precision, recall, and F1 of the best model of each kind (from section ??) on the dev and test sets after training. You may report results from a model trained on a subset of the training data. Try to use at least a 1000 sentences. Make sure to report all of your assumptions in the report, including hyperparameter values.
2. To demonstrate that you have implemented minibatching correctly, report the training time for one epoch for at least two different batch sizes. You may use a subset of the data.
3. Submit the outputs of the best model for both the dev and test sets, for each kind of model (1, 3, 4) you implemented.
4. Look at the output on the dev set and compare to the gold labels. What are your observations about the errors your model makes?

Submission Instructions

Submit a zip file (A4 .zip) on Canvas, containing the following:

- **Output:** Submit the output of your code on the dev and test sets if it is asked for in each section.
- **Code:** Your code should be implemented in Python 3, and needs to be runnable. Submit your code together with a neatly written README file to instruct how to run your code with different settings. We assume that you always follow good practice of coding (commenting, structuring), and these factors are not central to your grade. However, please provide well commented code if you want partial credit. If you have multiple files, provide a short description in the preamble of each file.
- **Report:** As noted above, your writeup should be four pages long, or less, in PDF (one-inch margins, reasonable font sizes). Include your name at the top of the report. Part of the training we aim to give you in this class includes practice with technical writing. Organize your report as neatly as possible, and articulate your thoughts as clearly as possible. We prefer quality over quantity. Do not flood the report with tangential information such as low-level documentation of your code that belongs in code comments or the README. Similarly, when discussing the experimental results, do not copy and paste the entire system output directly to the report. Instead, create tables and figures to organize the experimental results.

A Appendix: Implementation of Minibatching

- **Masking the inputs**

Generally, LSTMs can accept variable length inputs. However, to perform minibatching using DataLoader, all the inputs in a batch need to be of the same length. This is done by a process called **masking**, where an input sequence is *padded* with a padding token to make it of the same length as the maximum length input in the batch.

When preprocessing the data, we convert the input sequence into a sequence of indices. A sentence after preprocessing may look like this: [12, 1, 5, 6]

If the max sentence length in a batch is 10, and the padding_id is 0, then the padded version of the sentence would be: [12, 1, 5, 6, 0, 0, 0, 0, 0, 0]

Since this is a sequence labeling task, each word in the input sentence is associated with a label. Therefore, we will need to pad both the input sentence and the label sequence during training. Before the input goes into the LSTM, we will need to unpad the sequence. In order to be able to do this, we store the lengths of the sentences before we pad them. The *collate_fn* parameter in DataLoader lets us write a customized method to process the batches. To do the padding, and subsequent unpadding, you may use the *pad_sequence()*, *pack_padded_sequence()* and *pad_packed_sequence()* methods defined in torch.nn.utils.rnn

- **Calculating batch loss**

In the [reference code](#), the loss is computed for each gold and predicted sequence inside the main training loop. In the mini-batched version, we compute the loss over each batch inside the main training loop. Here are some required changes to the reference code to bear in mind:

- *_get_lstm_features()*: *_get_lstm_features()* computes the lstm features. However, now, we have padded the inputs, therefore, we unpack the sequence before feeding it into the lstm, and then pack the output of the lstm. Use *pack_padded_sequence()* and *pad_packed_sequence()*. Note that the lstm_features tensor is now batch_size x maxSeqLen x tagset_size, different from sentence.length x tagset_size, like in the reference code.
- *forward_alg()*, *_viterbi_decode()*, *_score_sentence()*: *forward_alg()* now compute the scores over each batch, instead of each sentence. The scores for all sentences in a batch are added to get the total score for a batch. The padded values do not contribute to the scores, therefore a masked version of the input is required to do these computations. Similarly, *_score_sentence()* gives the score of a provided tag sequence for a batch of sentences.

To handle and manipulate your tensors, you may need to use some of the following tensor operations/methods. Further documentation at <https://pytorch.org/docs/stable/tensors.html>

- **view() and reshape()**

tensor.view() or tensor.reshape() returns a new tensor with the same data and number of elements as the old one, but a different shape specified in the parameters. Refer to the examples [here](#). -1 is used as a parameter to derive the length of one of the dimensions based on the other dimensions and the number of elements, and only one dimension may be specified as -1.

- **squeeze(), unsqueeze()**

squeeze() and unsqueeze() are reverse operations of each other. tensor.squeeze(input) returns a tensor with all the dimensions of input of size 1 removed. tensor.unsqueeze(input, dim) returns a new tensor with a dimension of size 1 inserted at the specified position (dim). Documentation for [squeeze](#) and [unsqueeze](#)

- **contiguous()**

tensor.contiguous() makes a copy of tensor so the order of the elements would be the same as if a tensor of the same shape was created from scratch.

- **transpose()**

transpose(input, dim0, dim1) returns a tensor that is a transposed version of input. The dimensions dim0 and dim1 are swapped.

- **permute()**

tensor.permute(*dims) rearranges the original tensor according to the desired ordering (*dims) and returns a new multidimensional rotated tensor

- **flatten()**
flatten(input, start_dim=0, end_dim=-1) flattens a contiguous range of dims in input. Examples [here](#)
- **expand()**
tensor.expand() returns a new view of the self tensor with singleton dimensions expanded to a larger size. Passing -1 as the size for a dimension does not change the size of that dimension.
- **scatter_() and gather()**
scatter_() and gather() are reverse operations of each other. gather(input, dim, index) gathers values of indices along an axis specified by dim. scatter_(dim, index, src) writes all values from the tensor src into self at the indices specified in the index tensor.
- **masked_select()**
masked_select(input, mask) returns a new 1-D tensor which indexes the input tensor according to the boolean mask (which is a BoolTensor).
- **masked_scatter()**
tensor.masked_scatter_(mask, source) copies elements from source into tensor at positions where the mask is one.
- **eq(), ge()**
tensor.eq(other) or tensor.ge(other) computes element-wise equality or $\geq$ for tensor and other, where other can be a number or a tensor whose shape is broadcastable with the first argument..